

RV32I-ISA-Decoder

1. Core Implementation: `instruction.hpp` and `decoder.cpp`

The RV32I-ISA-Decoder is a fully functional, custom-built Instruction Set Architecture decoder designed to accurately parse and disassemble the base RISC-V 32-bit integer instruction set (RV32I). The implementation strictly adheres to the official RISC-V specifications, ensuring that every 32-bit raw instruction is flawlessly translated into its corresponding assembly representation.

- **`instruction.hpp`:** This header file forms the backbone of the decoding logic. It encapsulates the Instruction structure, which handles the complex bitwise operations required to extract opcodes, registers (`rd`, `rs1`, `rs2`), and function codes (`funct3`, `funct7`). It also accurately reconstructs and sign-extends immediate values across all instruction formats (R, I, S, B, U, and J-Types) according to the RISC-V standard.
- **`decoder.cpp`:** This file serves as the main driver. It utilizes the ELFIO library to open and read real `.elf` executable files, specifically locating and extracting the `.text` section. It then processes the binary data in 4-byte chunks, feeding each raw instruction into the decoder logic to print the human-readable assembly code alongside its memory address.

2. Output Verification (Toolchain vs. Custom Decoder)

To validate the accuracy of the decoder, the output is directly compared against standard compiler outputs. Below is a visual comparison between the assembly generated by the official RISC-V GNU Toolchain (`.S` file or `objdump`) and the terminal output produced by our custom decoder running on the same `.elf` binary.

Snapshot from test.S generated from RISC-V GNU Toolchain vs custom RV32I decoder

```
.type __Z11my_functionv, @function
__Z11my_functionv:
.LFB0:
.cfi_startproc
    addi    sp,sp,-32
.cfi_def_cfa_offset 32
    sw     ra,28(sp)
    sw     s0,24(sp)
.cfi_offset 1, -4
.cfi_offset 8, -8
    addi    s0,sp,32
.cfi_def_cfa 8, 0
    sw     zero,-20(s0)
    lw     a5,-20(s0)
    addi    a5,a5,1
    sw     a5,-20(s0)
    nop
    lw     ra,28(sp)
.cfi_restore 1
    lw     s0,24(sp)
.cfi_restore 8
.cfi_def_cfa 2, 32
    addi    sp,sp,32
.cfi_def_cfa_offset 0
    jr     ra
.cfi_endproc
```

0x10094	0xfe010113	ADDI sp, sp, -32
0x10098	0x00112e23	SW ra, 28(sp)
0x1009c	0x00812c23	SW s0, 24(sp)
0x100a0	0x02010413	ADDI s0, sp, 32
0x100a4	0xfe042623	SW zero, -20(s0)
0x100a8	0xfec42783	LW a5, -20(s0)
0x100ac	0x00178793	ADDI a5, a5, 1
0x100b0	0xfef42623	SW a5, -20(s0)
0x100b4	0x00000013	ADDI zero, zero, 0
0x100b8	0x01c12083	LW ra, 28(sp)
0x100bc	0x01812403	LW s0, 24(sp)
0x100c0	0x02010113	ADDI sp, sp, 32
0x100c4	0x00008067	JALR zero, ra, 0x0
0x100c8	0xfb010113	ADDI sp, sp, -80
0x100cc	0x04112623	SW ra, 76(sp)
0x100d0	0x04812423	SW s0, 72(sp)
0x100d4	0x05010413	ADDI s0, sp, 80
0x100d8	0x00f00793	ADDI a5, zero, 15
0x100dc	0xfef42623	SW a5, -20(s0)
0x100e0	0x01400793	ADDI a5, zero, 20
0x100e4	0xfef42423	SW a5, -24(s0)
0x100e8	0xfec42703	LW a4, -20(s0)

3. Build System: Makefile

For the development stage, the project utilizes a standard Makefile to manage the compilation and linking process. The Makefile simplifies the build workflow by defining clear targets:

- **make all:** Compiles the core decoder.cpp and links the necessary dependencies to generate the main executable.
- **make test:** Compiles the test files and links them with the gtest and pthread libraries to generate the unit testing binary.
- **make test_lit:** Automates the execution of the end-to-end regression tests using the command line.

4. Testing & CI/CD Pipeline

The testing stage is strictly divided to ensure both functional accuracy and system stability over time, utilizing Google Test for unit tests and LIT for regression tests.

- **Unit Testing (gtest):** Google Test is used to isolate and test the internal mathematical decoding logic. Parameterized tests feed raw hex values into the Instruction struct to verify that the extraction of registers and complex immediate offsets (such as negative branching values) is mathematically flawless.
- **Regression Testing (LIT):** The LLVM Integrated Tester (LIT), paired with FileCheck, is used for end-to-end regression testing. This ensures that as the codebase evolves, the decoder's terminal output for real .elf files remains perfectly matched with a known, stable baseline output.
- **CI/CD Integration:** The project supports a Continuous Integration/Continuous Deployment (CI/CD) pipeline via GitHub Actions to automate the build and test stages. Upon every push to the repository, a cloud environment automatically compiles the code and runs both the gtest and LIT suites, guaranteeing that no code regressions are merged into the main branch.